

End of Semester Report - Intrusion Detection Systems

Daniel Broyd

September 8, 2025

1 General Information

We target lateral movement, the chained pivot across internal hosts to expand access and privilege. Lateral movement is identity-centric (focuses on identity to access critical resources) and largely 'living off the land': the intruder investigates quietly, reuses or elevates credentials (often via token manipulation), prefers built-in administrative channels over malware, and stages only briefly before cleaning up. In logs this comes through as a time-linked behavior rather than a single spike: new or unusual process-start chains, with the command lines and parents telling the story, paired with occasional process-to-process memory access against sensitive targets, and per-process east-west connections to first-time peers as the attacker reaches across machines. Along the way, remote execution leaves the soft footprints of named pipes being created and connected, quick file writes and deletes in staging paths, and small registry changes used to trigger or persist execution. What ultimately distinguishes the attack is the sequence: a management shell or script host spawns an admin tool, resolves a host, talks to it, opens a pipe, drops a helper, runs it, and then tidies up, forming a cross-host path that a learning-based IDS can separate from routine administration.

2 Dataset and Features

2.1 Dataset Details

For the experiments, We used the **LMD-2023 (Lateral-Movement Dataset)**. The dataset is publicly available on GitHub, here (Temporarily moved to Google Drive here). LMD-2023 was produced by actively re-executing a set of lateral-movement techniques on a Windows domain testbed composed of both physical and virtual machines.

Specifically, the authors repeated the original LMD-2022 attacks several times and extended the corpus by including six recent techniques (such as Log4Shell, Follina, SM-BGhost, and Zerologon), resulting in traces of 15 distinct lateral movement methods captured by Microsoft Sysmon. The dataset contains both raw Sysmon/EVTX logs and preprocessed CSV files.

For labeling, the dataset distinguishes between three classes: *Normal*, *EoRS* (Exploitation of Remote Services), and *EoHT* (Exploitation of Hashing techniques). The labeling process was conducted with rules and scripts that map Sysmon events into these categories, as detailed by the dataset authors.

The dataset structure is organized as follows: each version includes (a) a labeled (normal, attack) CSV file with 93 features, and (b) separate subset partitions CSV files

isolating the three traffic categories (Normal, EoRS, and EoHT). In short, the logical layout is:

- LMD-2023/<version>/{Labeled}.CSV
- LMD-2023/<version>/subsets/{Normal, EoRS, EoHT}.CSV

2.2 Extracted features and feature-extraction process

All ML methods used the same compact feature set extracted from the LMD-2023 CSV export.

- **Computer:** Hostname that emitted the Sysmon event. This identifies which machine produced an event (useful because lateral movement is host-to-host) and was treated as a categorical feature (one-hot encoded, ignores unknowns) so models can learn host-specific baselines and spot unusual host activity.
- **DestinationPortName:** Resolved service name for the network destination (e.g., `http`, `smb`, `rdp`). It directly reflects service/protocol usage common in lateral techniques and was used as a categorical feature (one-hot) to detect atypical service access.
- **EventID:** Sysmon event type identifier (e.g., process creation, network connection). This nominal field signals which telemetry type generated the record and was one-hot encoded so the models focus on relevant event families (network vs process vs registry).
- **EventRecordID:** Sequential record number from the Windows event log. Used as a numeric feature to preserve ordering and detect bursts or abnormal densities of events when combined with time information; it was median-imputed if missing and Min-Max scaled.
- **Execution_ProcessID:** Process identifier associated with the executed action. This links events to the originating process (helpful to correlate suspicious process-driven network activity) and was treated as a numeric ID, imputed and scaled for modelling.
- **Initiated:** Boolean-ish flag indicating whether a network connection was initiated locally. This inbound/outbound distinction is important for spotting pivoting behaviour and was encoded as a binary/categorical feature.
- **ProcessId:** Operating-system process id for the event. Like **Execution_ProcessID**, it ties events to a process instance (useful for detecting unusual process behavior) and was included as a numeric, imputed and scaled feature.
- **SourceIsIpv6:** Flag indicating whether the source IP is IPv6. It separates IPv6 vs IPv4 activity (different baselines and potential attack channels) and was encoded as a binary feature.
- **SystemTime** (and derived components): Original event timestamp; we convert it to datetime and derive components (year, month, week, day-of-week, hour). Time-derived buckets capture diurnal/weekly patterns and event sequencing (attacks often occur at atypical times) and those components are used as categorical/time-bucket features (one-hot or discrete bins).

Preprocessing summary: We load the labelled CSV, select the features above, convert `SystemTime` to datetime and extract time components, one-hot encode categorical/time-bucket fields (`Computer`, `DestinationPortName`, `EventID`, `Initiated`, `SourceIsIpv6`, time components), impute numeric ID fields (`EventRecordID`, `ExecutionProcessID`, `ProcessID`) and apply Min-Max scaling to numeric columns. No automated feature-selection algorithm was used; the chosen compact set reflects a practical subset of the LMD-2023 exporter fields that retain event, process and network semantics relevant for lateral-movement detection - while matching the original paper, maintaining consistency and allowing better result comparison.

Feature Importance To learn more about the feature importance, Figure 1 shows the Random Forest feature-importance bar plot produced in the implementation (top 20 features). the results show the model relies heavily on event-order / host indicators (e.g., `EventRecordID` and host one-hot features) together with network/event-type signals and simple time/process fields. The prominence of `EventRecordID` and host indicators alongside network/event-type and time/process features indicates that the compact set captures complementary signals - event-order and host context (where/when anomalous sequences occur) together with direct network/process indicators (what actions happened).

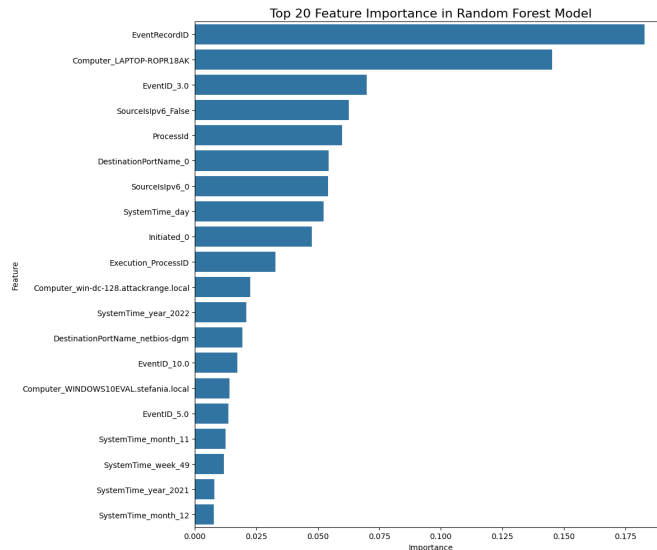


Figure 1: Random Forest — top feature importances.

3 Implementation And Results

3.1 Variational Auto-Encoder

- **Implementation and parameters.**

We model normal host activity with a stacked Variational Auto-Encoder (VAE) and flag anomalies by reconstruction error.

- **Architecture:** The encoder is a feed-forward network (**deep** configuration) with linear layers $d \rightarrow 50 \rightarrow 30 \rightarrow 20$, BatchNorm and ReLU; it outputs

$(\mu, \log \sigma^2) \in \mathbb{R}^{10}$ for a 10-dimensional latent. The decoder mirrors this ($10 \rightarrow 20 \rightarrow 30 \rightarrow 50 \rightarrow d$) and ends with a sigmoid to reconstruct to $[0, 1]$.

- **Loss Function:** We optimize the following Loss:

$$\mathcal{L}(x) = \underbrace{\text{BCE}(\hat{x}, x)}_{\text{reconstruction}} + \underbrace{D_{\text{KL}}(q_\phi(z | x) \| \mathcal{N}(0, I))}_{\text{regularization}},$$

Where:

$$\begin{aligned} \text{BCE}(\hat{x}, x) &= - \sum_i [x_i \log(\hat{x}_i) + (1 - x_i) \log(1 - \hat{x}_i)] \\ D_{\text{KL}}(q_\phi(z | x) \| \mathcal{N}(0, I)) &= \frac{1}{2} \sum_j (\exp(\sigma_j) - 1 - \sigma_j + \mu_j^2), \end{aligned}$$

using the reparameterization trick $z = \mu + \sigma \odot \epsilon$, $\epsilon \sim \mathcal{N}(0, I)$.

- **Training Parameters:** Training uses Adam (lr= 10^{-3} , weight decay = 10^{-5}), batch size 50, for 20 epochs, with checkpointing. At inference we score each sample by MSE reconstruction error (for comparability with our stacked AE) and apply a fixed threshold.
- **Hyperparameter Tuning:** We compared the **simple** vs. **deep** encoder/decoder layouts and selected **deep** for more stable loss and higher validation F1. We varied latent size $\{8, 10, 16\}$ and found 10 balances compression (precision) with expressivity (recall). Learning rates in $\{10^{-4}, 10^{-3}\}$ behaved as follows: 10^{-3} converged faster without instability. Finally, percentile thresholds on benign validation error (95, 97.5, 99) trade precision for recall; we report the 99th percentile operating point used in the implementation.

- **Training and testing details.**

- Only benign records are used to fit the VAE, Since we expect the model to perform poorly on samples that are far from benign sample distribution; labels are used after training, only for threshold selection and evaluation.
- The benign split is 80/20 (train/test), yielding **1,306,322** train and **326,581** benign test samples.
- The attack set used for evaluation contains **511,105** samples.
- The anomaly threshold is fixed to the 99th percentile of benign test reconstruction error, which in our run means: $\mathcal{L} = 0.040$.

- **Detection results**

At the 99th-percentile operating point the VAE attains:

- **Accuracy:** 96.01%
- **Precision:** 99.33%
- **Recall:** 94.10%
- **F1-Score:** 96.64%

Figure 2 shows the VAE confusion matrix.

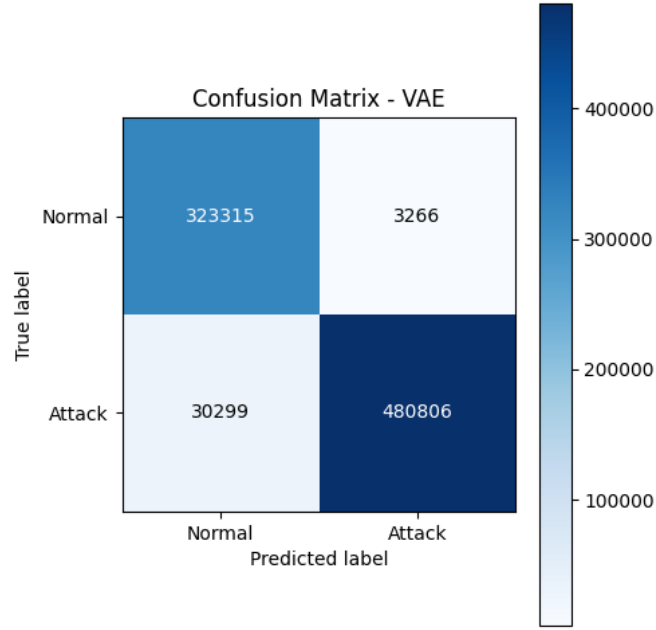


Figure 2: Confusion matrix for the VAE at the 99th-percentile threshold.

- Error analysis

- False Positive:

False Positive example	
Computer	WINDOWS10EVAL.stefania.local
DestinationPortName	0
EventID	1
EventRecordID	36011
Execution_ProcessID	0
Initiated	0
ProcessId	692
SourceIsIpv6	0
Hour	11

Correctly-classified benign (for comparison):

Correctly-classified benign example	
Computer	LAPTOP-ROPR18AK
DestinationPortName	netbios-dgm
EventID	3
EventRecordID	2219
Execution_ProcessID	0
Initiated	False
ProcessId	4
SourceIsIpv6	False
Hour	16

The false-positive row is a benign process-creation event (`EventID=1`) with no

network signals (`DestinationPortName=0`, `Initiated=0`) and missing parent/process-context (`Execution_ProcessID=0`). The Random Forest places strong weight on `EventRecordID` and host one-hot indicators - therefore we assume that these features are highly important for the VAE as well (we can't know for sure, since VAE doesn't have a natural feature importance scale unlike the RF). Because this event occurs on

`WINDOWS10EVAL.stefania.local` and within an `EventRecordID` range correlated with attacks in the training set, the model falls back to those contextual cues and flags it. The correctly-classified benign example differs in host, event-type and record-range (e.g., `EventID=3`, non-empty `DestinationPortName`), which gives the classifier clearer normal-pattern signals and prevents a false alarm.

– **False Negative:**

False Negative example	
Computer	<code>WINDOWS10EVAL.stefania.local</code>
DestinationPortName	0
EventID	1
EventRecordID	26205
Execution_ProcessID	0
Initiated	0
ProcessId	704
SourceIsIpv6	0
Hour	14

Correctly-classified attack (for comparison):

Correctly-classified attack example	
Computer	<code>win-host-987.attackrange.local</code>
DestinationPortName	-
EventID	3
EventRecordID	59037796
Execution_ProcessID	0
Initiated	True
ProcessId	5272
SourceIsIpv6	False
Hour	9

The false-negative event is labelled as attack in the dataset but is a process-creation record without explicit network indicators (`DestinationPortName=0`, `Initiated=0`) and with missing parent-process context (`Execution_ProcessID=0`). As we mentioned earlier, the Random Forest relies heavily (therefore we assume the VAE does the same) on network/event-type signals (e.g., `EventID=3`, non-zero `DestinationPortName`) and on host/record-range correlations; when these stronger signals are absent the event's numeric/time signature closely matches benign process events and the classifier predicts normal. The correctly-classified attack example, by contrast, contains a clear network connection pattern (`EventID=3` and `Initiated=True`) and a distinct record/host context, which the model can pick up reliably.

3.2 Random Forest

- **Parameters (hyperparameters used / tuned)**

The Random Forest classifier configuration and the hyperparameter grid explored in the implementation were as follows:

- **Base settings (used in pipeline):**

- * **random_state** = 42: reproducibility.
- * **class_weight** = **balanced_subsample**: compensate label imbalance per tree (preferred here because bootstrap sampling is used).
- * **n_jobs** = 1 (classifier): avoid nested-parallelism conflicts while GridSearch runs in parallel.

- **Grid-search hyperparameters (grid_search evaluated):**

- * **n_estimators** $\in \{200, 300\}$: number of trees in the forest.
- * **max_depth** $\in \{20, \text{None}\}$: maximum depth of each tree (None = unlimited).
- * **min_samples_split** $\in \{2, 5\}$: minimum samples required to split an internal node.

Why those choices & observed effects: we explicitly avoided blind defaults by running a **GridSearchCV** (cross-validation), (scoring='f1') over the small grid above using

StratifiedKFold(n_splits=3, shuffle=True, random_state=42) and inspecting mean CV F1 for each combination.

We use F1 since it balances the importance of precision and recall, and is preferable to accuracy for class-imbalanced datasets.

The observed trends were: increasing the number of estimators from 200 to 300 yielded a small, consistent improvement in mean CV F1; allowing unlimited depth (**max_depth=None**) produced better CV-F1 than restricting depth to 20 (but only a small margin); and choosing the smaller **min_samples_split=2** gave a minor F1 advantage over 5.

In short: deeper trees and slightly more trees improved fit marginally; the effects were measurable but numerically small (differences on the order of a few 10^{-4} in mean CV F1 in the results). **Representative grid results (CV summaries):**

n_estimators	max_depth	mean CV F1 (approx.)
300	None	0.99633
200	None	0.99632
300	20	0.99591
200	20	0.99589

the table above summarises the main observed trend: marginal gains for deeper / more trees.

- **Training and testing details**

- **Train/test split:** the dataset was split with `train_test_split(..., test_size=0.2, stratify=y, random_state=42)` (80% train, 20% test, stratified by label).
- **Preprocessing (fit on training only):** a `ColumnTransformer` was used with two small pipelines:
 - * **Categorical / OHE pipeline:** `SimpleImputer(strategy='most_frequent')` followed by `OneHotEncoder(handle_unknown='ignore')` for categorical and derived time-bucket fields. We ignore unknowns to effectively one-hot features that can be vastly different, for example Computer - we don't want an unknown computer name to have an impact on the decision of the network.
 - * **Numeric / MinMax pipeline:** `SimpleImputer(strategy='median')` followed by `MinMaxScaler()` for numeric ID/time features.

The preprocessor is *fitted only* on `X_train` and then used to transform `X_test` (no leakage).

- **Model pipeline and search:** an `ImbPipeline` containing the `RandomForest` classifier was tuned with `GridSearchCV` using `scoring='f1'`, `cv=StratifiedKfold(n_splits=10, shuffle=True, random_state=42)`, and `n_jobs=-1` for the grid search.
- **Final evaluation:** we save and then load the best grid-search estimator and applies it to `X_test_processed`. Test metrics are computed with `accuracy_score` and `precision_recall_fscore_support(average='binary')`, plus the confusion matrix.

• Detection results

Final test metrics and the confusion matrix for the best-found Random Forest:

- **Accuracy:** ≈ 0.9984
- **Precision:** ≈ 0.9984
- **Recall:** ≈ 0.9950
- **F1-score:** ≈ 0.9967

Figure 3 shows the Random Forest Confusion Matrix.

• Error analysis

- **False Positive:**

False Positive example	
Computer	WINDOWS10EVAL.stefania.local
DestinationPortName	0
EventID	1
EventRecordID	36011
Execution_ProcessID	0
Initiated	0
ProcessId	692
SourceIsIpv6	0
Hour	11

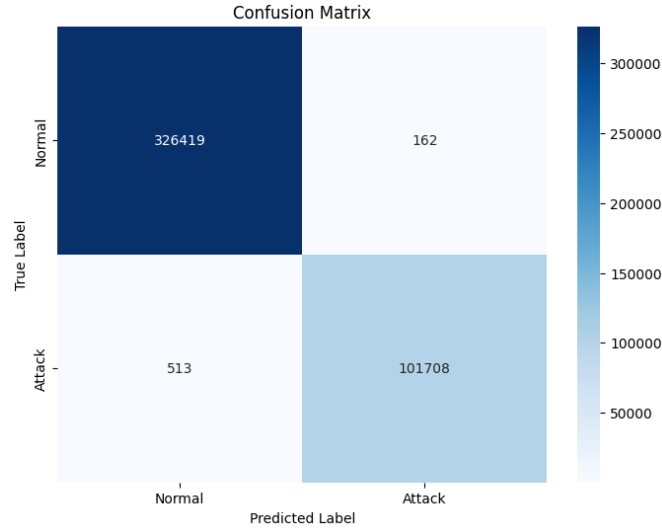


Figure 3: Confusion matrix for the Random Forest.

Many attack-labelled training rows share host/time/record-index correlations (the model’s top importances include **EventRecordID** and specific **Computer_*** one-hot features). This row is a benign process-creation event (**EventID**=1) with no network signals (**DestinationPortName**=0, **Initiated**=0). Lacking stronger per-event indicators (command-line, parent image, network connection), the classifier falls back to correlated signals (host and record-range) and thus produces a false alarm.

– **False Negative:**

False Negative example	
Computer	WINDOWS10EVAL.stefania.local
DestinationPortName	0
EventID	1
EventRecordID	26205
Execution_ProcessID	0
Initiated	0
ProcessId	704
SourceIsIpv6	0
Hour	14

This attack event is also a process-creation record that lacks explicit network indicators and has missing parent-process/execution-process information (**Execution_ProcessID**=0). The model relies strongly on network-event features (**EventID**=3 / **DestinationPortName** etc.) and on host/time/process-id correlations; when those stronger signals are absent and the event’s numeric/time signature closely matches benign examples, the classifier cannot distinguish it and labels it normal.

Model	Accuracy	Precision	Recall	F1-score
Random Forest (supervised)	99.84%	99.84%	99.50%	99.67%
Stacked Auto-Encoder (AE, unsupervised)	98.00%	99.35%	97.37%	98.35%
Variational Auto-Encoder (VAE, unsupervised)	96.01%	99.33%	94.10%	96.64%

Table 1: Test-set performance metrics for the three models.

4 Model Comparison

4.1 Results

The Random Forest strongly outperforms both reconstruction-based detectors because it is a supervised discriminative model trained on labelled attack and benign samples; with the same training volume it can learn explicit decision boundaries and exploit label-conditioned, fine-grained cues (host-specific one-hot patterns, EventRecordID ranges, event-type combinations) that reconstruction methods cannot use, which explains the near-perfect precision and very high recall on the test set.

Both the AE and the VAE are unsupervised (trained only on benign data) and therefore rely on reconstruction error to separate anomalies. This explains their generally high precision (benign patterns are well learned) but lower recall relative to the supervised RF: attacks that resemble benign patterns (under the compact feature set) can be reconstructed well and thus have low reconstruction error and get missed. Between the two unsupervised methods the AE outperformed the VAE in our runs: the deterministic AE produced tighter, more specific reconstructions of the benign training distribution, yielding larger reconstruction errors for many attack samples and improving separability; the VAE’s KL-regularized, probabilistic latent encourages smoother/generalized reconstructions which in this dataset reduced the reconstruction gap for some attacks, lowering recall and overall accuracy. Practically, this means RF is preferable when labelled attack data is available and high detection performance is required, while AE/VAE offer easier scalability (they only need benign records) with AE tending to be better when a detector that “overfits” benign samples improves anomaly separation, and VAE being preferable where smoother generalization and probabilistic modeling is desired (which is not our case - we want to overfit benign samples, and get higher error on less seen samples).

4.2 Cross-Model Error Comparison

To understand the models’ differing strengths and weaknesses, we analyzed how the supervised Random Forest (RF) classifies the specific log events that the unsupervised Stacked AE and VAE models misclassified.

The reason we focus on Random Forest rather than directly comparing AE and VAE is that the differences between AE and VAE results are often ambiguous without additional context. We also emphasize RF performance on AE and VAE errors, rather than the reverse, because RF achieves significantly higher accuracy.

The analysis reveals that the models’ errors are largely complementary, stemming from their different learning approaches (discriminative vs. generative). We’ve omitted fields with 0/no values from the demonstration, in order to focus on existing features.

Error Scenario Analysis

1. Agreement: RF & VAE both predict attack (False Positive)

Sample Details		Classification Outcome
Computer	WINDOWS10...	True Label: 0 (Normal)
EventID	13.0	VAE Prediction: 1 (Attack)
EventRecordID	34535.0	RF Prediction: 1 (Attack)
ProcessId	1296.0	
Hour	11	

The event is ambiguous, lacking clear network signals. The VAE flags it due to its anomalous feature combination, resulting in a high reconstruction error. The RF also misclassifies it because its contextual features (e.g., Computer and record ID) are strongly correlated with attack patterns learned during its supervised training.

2. Disagreement: RF predicts normal, correcting AE's False Positive

Sample Details		Classification Outcome
Computer	LAPTOP-RO...	True Label: 0 (Normal)
EventID	3.0	AE Prediction: 1 (Attack)
EventRecordID	5389.0	RF Prediction: 0 (Normal)
ProcessId	4.0	
Hour	10	

The AE likely flagged this sample due to an unusual combination of features that resulted in a high reconstruction error. The RF, however, has learned the specific decision boundaries distinguishing normal traffic and correctly identifies this pattern (e.g., this EventID on this host) as benign, overriding the simple anomaly signal. The standard business hour (10:00) reinforces the RF's classification of this event as routine administrative activity.

3. Agreement: RF & AE both predict normal (False Negative)

Sample Details		Classification Outcome
Computer	WINDOWS10...	True Label: 1 (Attack)
EventID	1.0	AE Prediction: 0 (Normal)
ProcessId	4560.0	RF Prediction: 0 (Normal)
EventRecordID	30041.0	
Hour	13	

This is a consistently difficult event type. It is an attack event (EventID=1) that strongly resembles normal process creation activity. Because it lacks distinct anomalous signals, the AE reconstructs it well (low error), and the RF, missing strongly discriminative features, classifies it based on its similarity to benign training samples.

4. Disagreement: RF predicts attack, correcting VAE's False Negative

The VAE failed to detect this attack because its feature values were not anomalous enough to produce a high reconstruction error, especially given the normal work hour (16:00). The

Sample Details		Classification Outcome
Computer	WIN-J23NI...	True Label: 1 (Attack)
EventID	3.0	VAE Prediction: 0 (Normal)
Initiated	False	RF Prediction: 1 (Attack)
ProcessId	1232.0	
Hour	16	

RF, benefiting from supervised training, identified a subtle, multi-feature pattern (e.g., this specific **ProcessId** and **EventID** on this host) that it learned is highly indicative of an attack, demonstrating its ability to find malicious patterns even within seemingly normal temporal contexts.

Analysis of Consistently Misclassified Events (Agreements)

The events that consistently fool multiple models are typically those that are inherently ambiguous within our constrained feature set. These are often **process creation events** (**EventID=1**) that lack overt network indicators or clear parent process context, with a similar appearance to benign samples.